

**КИЇВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ ТАРАСА ШЕВЧЕНКА**

Факультет комп'ютерних науки та кібернетики

ЗВІТ

до лабораторної роботи №2

із дисципліни «Бази даних та інформаційні системи»

на тему:

РОБОТА З СИСТЕМАМИ КЕРУВАННЯ БАЗАМИ ДАНИХ

Виконав

студент 2-го курсу

групи К-24

Дмитро ОСТАПЕНКО

Київ – 2024

Мета

Мета роботи – навчитися працювати з системами управління базами даних (СУБД) та створити графічний інтерфейс користувача для роботи з базою даних на певну тему.

Постановка задачі

Для виконання роботи потрібно не менше 5 взаємно зв'язаних та заповнених даними таблиць. У периферійних таблицях не менше 4-5 рядків, а у зв'язкових не менше 10 рядків.

Форми

1. Всі дані повинні вводитись, переглядатись та редагуватись лише через форми користувача.
2. При вводі чи редагуванні даних необхідно здійснювати контроль коректності (формату) вводу. Наприклад, при спробі користувача ввести текст у числове поле (чи поле дати) потрібно блокувати таку дію з відповідним повідомленням від вашої системи, а не від базової СУБД. інший варіант – блокувати введення некоректних даних в індикативне поле форми, тоді потрібні підказки на формі (постійні чи такі, які з'являються при необхідності) про бажаний формат вводу.
3. При відкритті бази даних повинна виводитись головна форма-меню, на якій присутні посилання на всі інші форми, запити, ввід, редагування, перегляд. На всіх інших формах повинна бути кнопка повернення до головного меню.

Запити

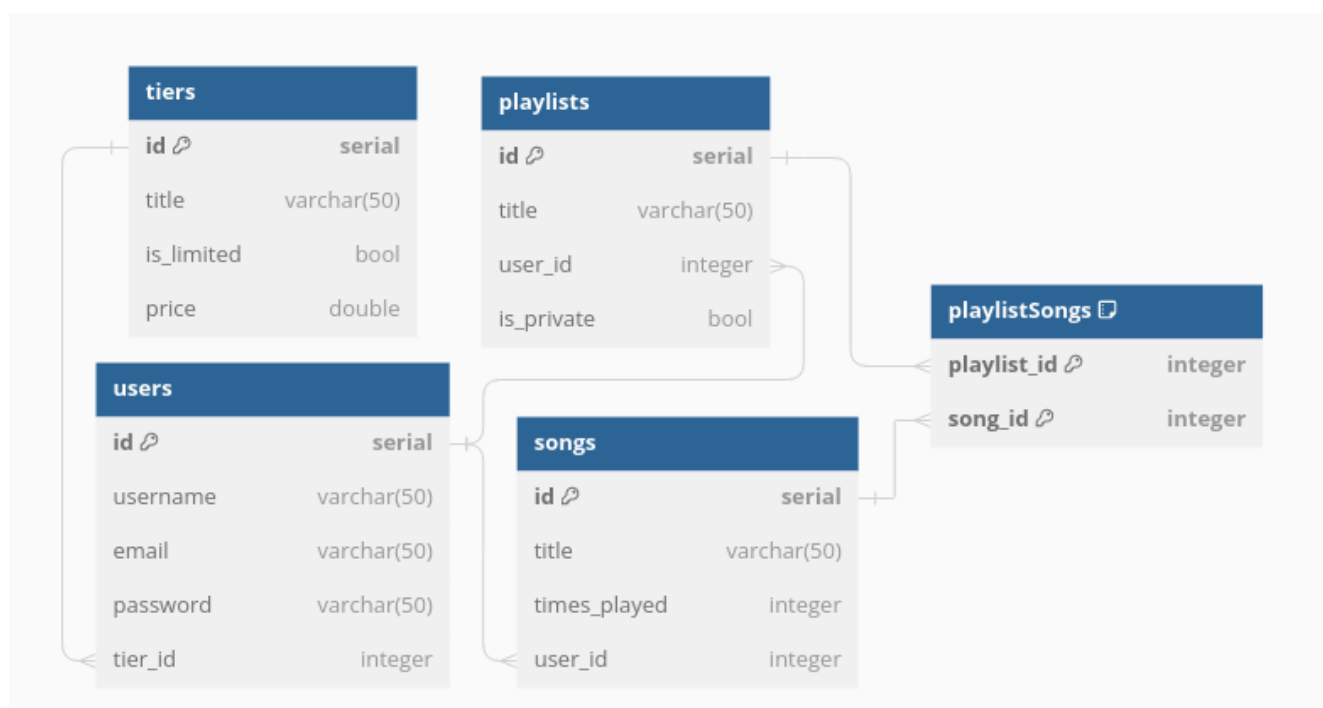
1. Повинно бути не менше 5 простих параметризованих запитів, що використовують більше однієї таблиці. Наприклад «Міста, в яких знаходиться принаймні один постачальник, що постачає принаймні одну деталь, вага котрої перевищує вказану користувачем величину». Вказана користувачем величина є параметром і в запиті позначається довільним ідентифікатором, наприклад, X. При виконанні запиту користувачеві буде запропоновано ввести значення параметру.

2. Повинно бути не менше 3 параметризованих запитів із множинними порівняннями. Наприклад, «імена постачальників, що постачають точно такі ж деталі, як і постачальник S1», «імена постачальників всіх деталей», «Пари номерів постачальників, що постачають однакову множину деталей»;

Виконання

Для реалізації завдань лабораторної роботи використовувалася технологія Tauri (фреймворк для побудови настільних/мобільних застосунків, що використовує мову програмування Rust та будує відображення на основі мови HTML), SvelteKit для побудови графічного інтерфейсу та PostgreSQL як СУБД.

Тема – платформа із музикою, альбомами та списками відтворення. Схему бази даних можна побачити на малюнку 1.



Мал. 1: Схема бази даних

Графічний інтерфейс користувача

Як уже зазначалося, графічний інтерфейс побудовано у форматі настільного застосунку (Додаток 2).

Особливості:

1. При наведенні на зовнішній ключ можна побачити більш зрозуміле поле, що відповідає запису з цим ключем (Додаток 2.1).
2. На верхній панелі є кнопка допомоги, при натисканні на яку відкривається вікно допомоги (Додаток 2.6), що містить інформацію про застосунок, автора та можливості.

3. Реалізовано гарячі клавіші, які можна знайти у вікні допомоги.
Зокрема, натиснувши Ctrl+N, користувач відкриває вікно допомоги.
4. На верхній панелі присутня кнопка зміни кольорової схеми застосунку (світла або темна) (Додаток 2.7).
5. Графічний інтерфейс прив'язаний лише до схеми бази даних, тому може працювати із довільною базою даних, яка їй відповідає. Для цього користувач може сам вводити рядок підключення (Додаток 2.8)

Запити

Застосунок реалізує п'ять простих параметризованих запитів (1-5) та три запити з множинним порівнянням (6-8).

Частини SQL коду, які використовуються у запитах:

- Умова видимості списку відтворення:
 - порожня, якщо довільний;
 - “AND playlists.is_private”, якщо приватний;
 - “AND NOT playlists.is_private”, якщо публічний.
 - Умова обмеженості тарифу:
 - порожня, якщо обмежений;
 - “NOT ”, інакше.
 - Умова
1. Отримати всі пісні, що були завантажені користувачем з ідентифікатором **ID**, містять щонайменше **N** прослуховувань та входять до (**публічного/приватного/довільного**) списку відтворення.

SQL:

```
SELECT * FROM songs WHERE user_id={ID} AND times_played >=
{N} AND EXISTS (SELECT id FROM playlists INNER JOIN
playlistSongs on playlists.id=playlistSongs.playlist_id WHERE
songs.id=playlistSongs.song_id {умова видимості списку
відтворення}) ORDER BY id;
```

- Отримати всі пісні, що були завантажені користувачем з ідентифікатором **UID** та входять до списку відтворення з ідентифікатором **PID**.

SQL:

```
SELECT * FROM songs WHERE user_id={UID} AND EXISTS(SELECT
* FROM playlistSongs WHERE song_id=songs.id AND
playlist_id={PID})) ORDER BY id;
```

- Отримати всіх користувачів, які завантажили щонайменше **N** пісень та мають (**обмежений/звичайний**) тарифний план.

SQL:

```
SELECT * FROM users WHERE EXISTS (SELECT id FROM tiers
WHERE id=users.tier_id AND {умова обмеженості тарифу} is_limited)
AND (SELECT COUNT(id) FROM songs WHERE user_id=users.id
LIMIT {N+1}) >= {N} ORDER BY id;
```

- Отримати всіх користувачів, що мають тарифний план із назвою **Title** та є власниками хоча б 1 списку відтворення, який містить не менше **N** пісень.

SQL:

```
SELECT * FROM users WHERE EXISTS (SELECT id FROM tiers
WHERE id=users.tier_id AND title='{Title}')) AND EXISTS (SELECT id
FROM playlists WHERE user_id=users.id AND (SELECT
COUNT(playlist_id) FROM playlistSongs WHERE playlist_id=playlists.id
LIMIT {N+1}) >= {N})) ORDER BY id;
```

- Отримати усі тарифні плани, які відповідають щонайменше **N1** користувачам, які є власниками щонайменше **N2** списків відтворення із (**приватною/публічною/довільною**) видимістю.

SQL:

```
SELECT * FROM tiers WHERE (SELECT COUNT(id) FROM users
WHERE (SELECT COUNT(id) FROM playlists WHERE user_id=users.id
{умова видимості списку відтворення}) >= {N2} AND
tier_id=tiers.id) >= {N1} ORDER BY id;
```

6. Отримати всі списки відтворення з
(приватною/публічною/довільною) видимістю, які містять усі пісні,
що були завантажені користувачем з ідентифікатором **ID**.

SQL:

```
SELECT * FROM playlists WHERE (SELECT COUNT(id) FROM songs
WHERE user_id={ID}) = (SELECT COUNT(id) FROM songs WHERE
user_id={ID} AND EXISTS(SELECT playlist_id FROM playlistSongs
WHERE song_id=songs.id AND playlist_id=playlists.id)) {умова
видимості списку відтворення} ORDER BY id;
```

7. Отримати списки відтворення, власником яких є користувач, що
завантажив не менше **N** пісень, а також які містять усі пісні зі списку
відтворення з ідентифікатором **ID**.

SQL:

```
SELECT * FROM playlists WHERE (SELECT COUNT(id) FROM songs
WHERE user_id=playlists.user_id) >= {N} AND (SELECT DISTINCT
COUNT(song_id) FROM playlistSongs WHERE playlist_id={ID}) <=
(SELECT COUNT(playlist_id) FROM playlistSongs WHERE
playlist_id=playlists.id AND EXISTS(SELECT song_id FROM
playlistSongs as X WHERE X.playlist_id={ID} AND
X.song_id=playlistSongs.song_id)) ORDER BY id;
```

8. Отримати пісні, які входять до кожного списку відтворення.

SQL:

```
SELECT * FROM songs WHERE (SELECT COUNT(id) FROM playlists)
```

```
= (SELECT DISTINCT COUNT(DISTINCT playlist_id) FROM  
playlistSongs WHERE song_id=songs.id) ORDER BY id;
```


Підсумки

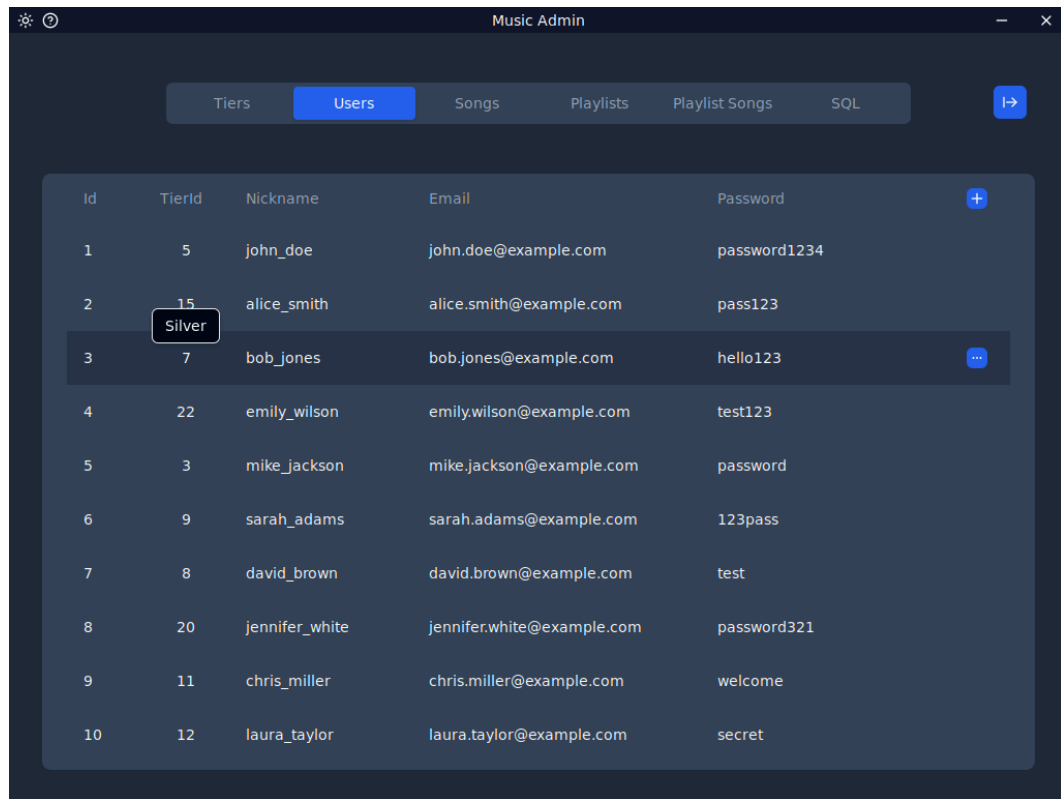
У ході виконання лабораторної роботи я реалізував зручний настільний застосунок із сучасним UI/UX, який дозволяє виконувати CRUD операції над базою даних, що задовольняє схему (Малюнок 1) та виконувати запити різного типу.

Завдяки роботі, що я виконав, я поглибив свої знання з SQL, отримав практичний досвід з новою технологією, значно покращив навички створення графічних інтерфейсів.

Додаток 1: вихідний код проєкту

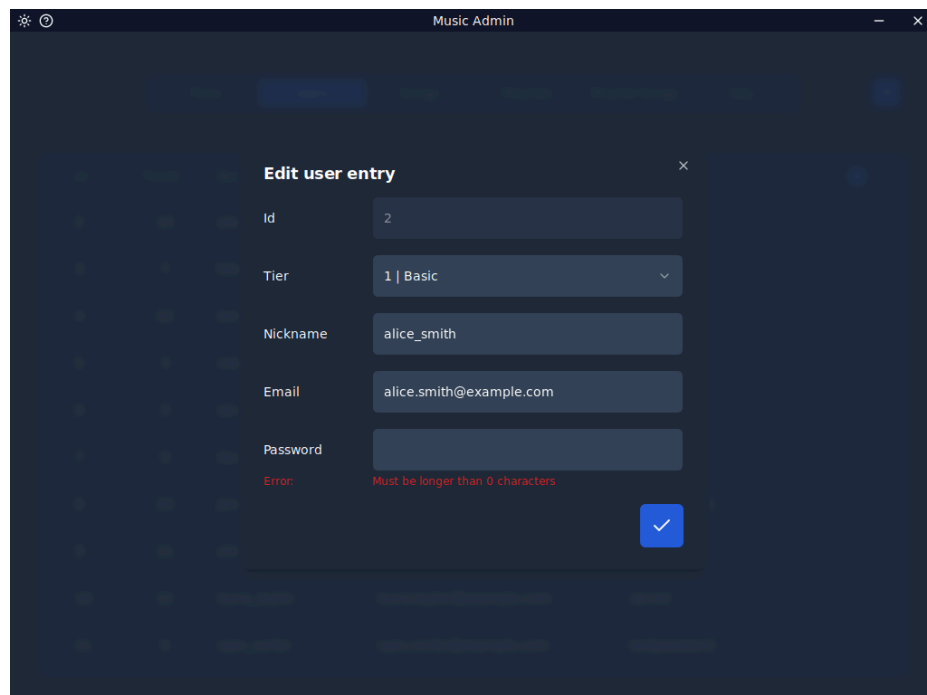
https://github.com/ODMyk/db_2

Додаток 2: графічний інтерфейс користувача



Id	TierId	Nickname	Email	Password
1	5	john_doe	john.doe@example.com	password1234
2	15 Silver	alice_smith	alice.smith@example.com	pass123
3	7	bob_jones	bob.jones@example.com	hello123
4	22	emily_wilson	emily.wilson@example.com	test123
5	3	mike_jackson	mike.jackson@example.com	password
6	9	sarah_adams	sarah.adams@example.com	123pass
7	8	david_brown	david.brown@example.com	test
8	20	jennifer_white	jennifer.white@example.com	password321
9	11	chris_miller	chris.miller@example.com	welcome
10	12	laura_taylor	laura.taylor@example.com	secret

2.1: Відображення таблиці та підказка для зовнішнього ключа



Edit user entry

Id: 2

Tier: 1 | Basic

Nickname: alice_smith

Email: alice.smith@example.com

Password:
 Error: Must be longer than 0 characters

✓

2.2: форма редагування запису

Create new User

Tier: 1 | Basic

Nickname: JohnDoe

Email: johndoe@example.com

Password:

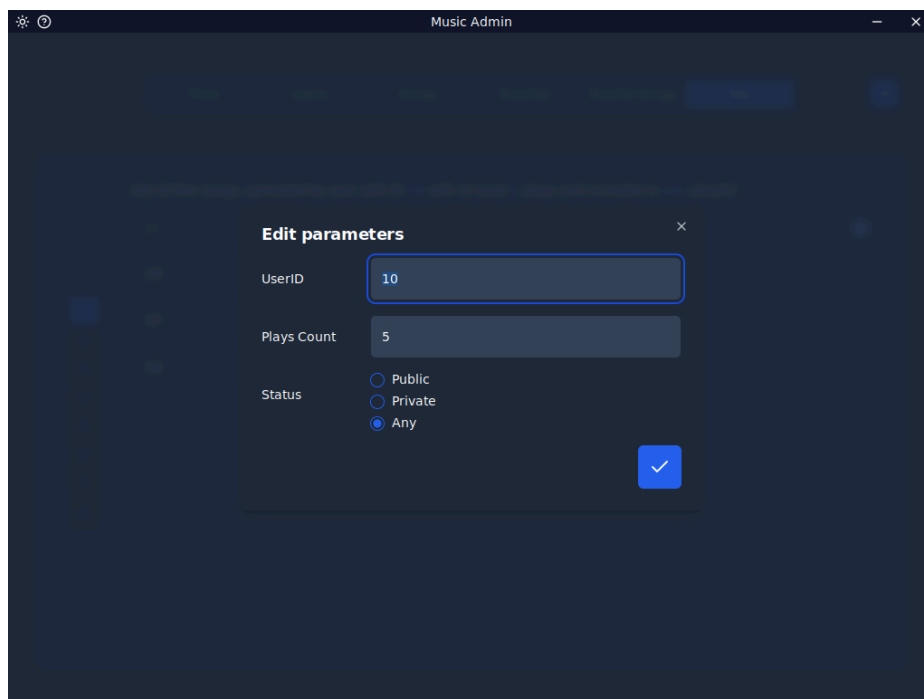
✓

2.3: форма створення запису

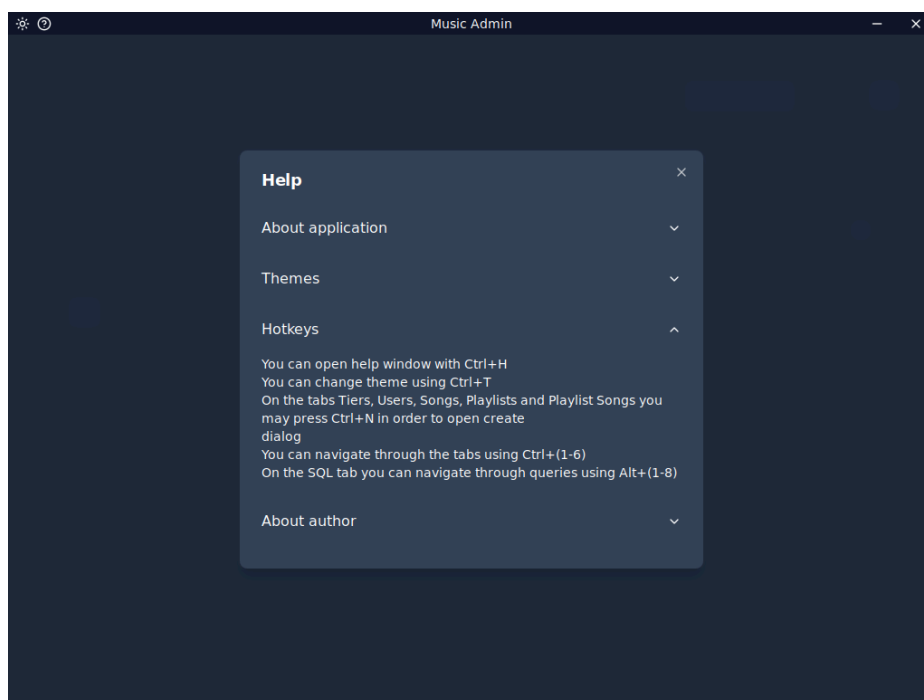
Get all the users, who uploaded at least 5 songs and have **limited** account tier

	Id	TierId	Nickname	Email	Password
1	3	7	bob_jones	bob.jones@example.com	hello123
2	38	15	grace_hughes	grace.hughes@example.com	testingtesting
3	43	9	dylan_jordan	dylan.jordan@example.com	testpassword1234
4	48	13	emily_garcia	emilygarcia@example.com	passwordtest1234
5					
6					
7					
8					

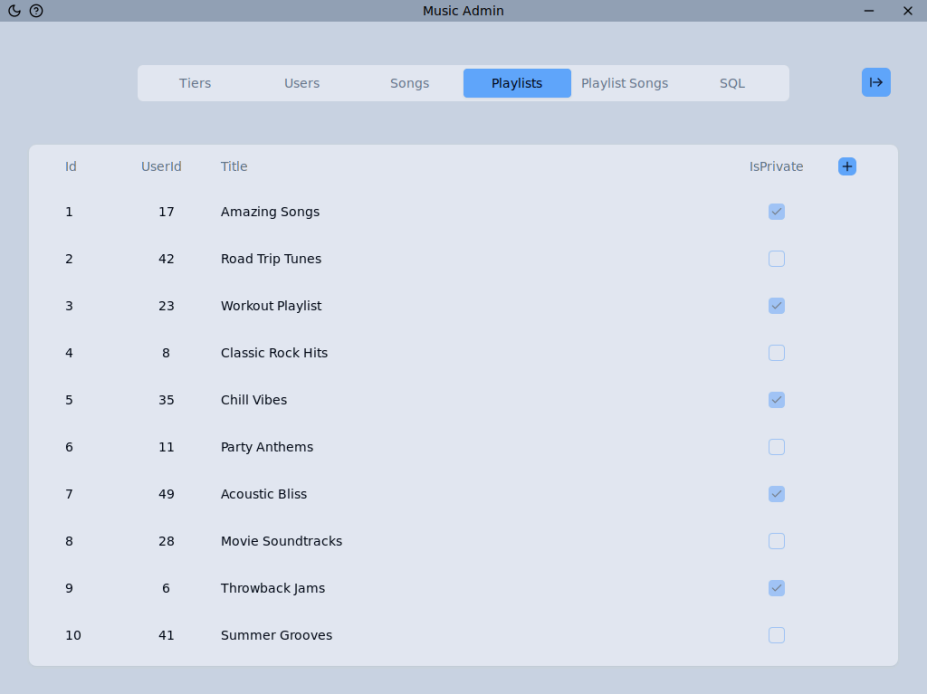
2.4: Вкладка запитів



2.5: Форма для зміни параметрів запиту

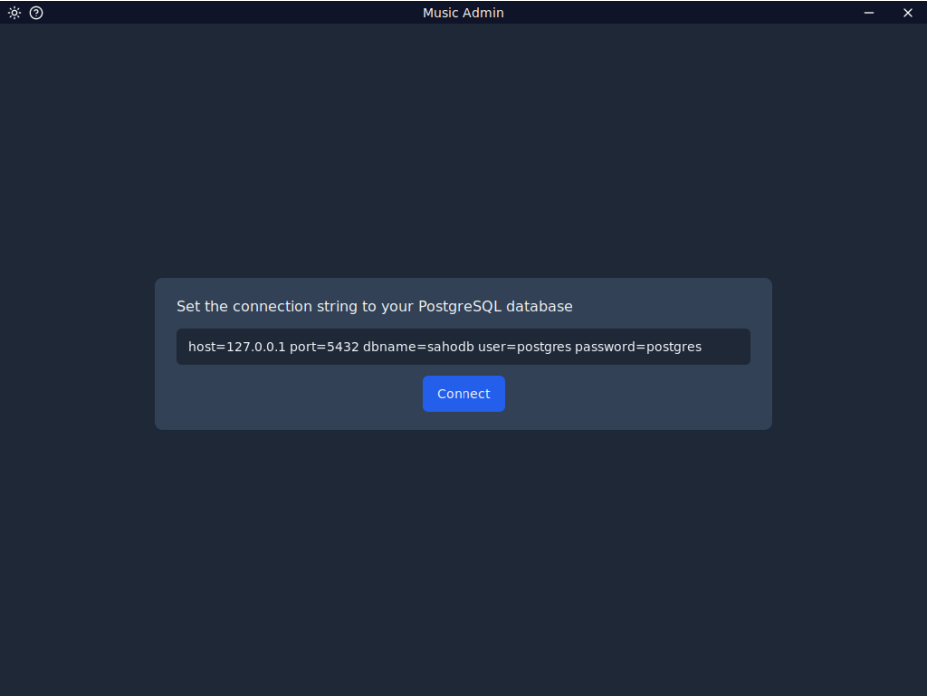


2.6: Вікно допомоги



Id	UserId	Title	IsPrivate
1	17	Amazing Songs	☒
2	42	Road Trip Tunes	☐
3	23	Workout Playlist	☒
4	8	Classic Rock Hits	☐
5	35	Chill Vibes	☒
6	11	Party Anthems	☐
7	49	Acoustic Bliss	☒
8	28	Movie Soundtracks	☐
9	6	Throwback Jams	☒
10	41	Summer Grooves	☐

2.7: Світла кольорова схема



Set the connection string to your PostgreSQL database

host=127.0.0.1 port=5432 dbname=sahodb user=postgres password=postgres

Connect

2.8: Форма введення рядку підключення до бази даних